

Rapport final de projet — Crypto Bot

Auteurs : Jules Willard — Mikaël Jayet

Formation : DataScientest — Cursus Data Engineer / Data Scientist

Période : Octobre 2025 – Juin 2026

Soutenance : Juin 2026

Version : V3 — mise à jour mai 2026

SYNTHÈSE

Contexte et ambition

Dans le cadre de notre formation DataScientest, nous avons conduit de bout en bout un projet fil rouge autour des marchés crypto : de la définition du cahier des charges jusqu'au déploiement en production.

Triple objectif :

- créer *"un bot de trading basé sur un modèle de Machine Learning qui investira sur les marchés crypto"*
- démontrer notre capacité à définir un cahier des charges, à implémenter une solution technique complète et à assurer sa mise en production avec monitoring.

Nous avons structuré notre réponse autour d'une plateforme intégrée baptisée **Crypto Bot**, adressant trois personas identifiés en phase de discovery : Noah (trader indépendant), Sarah (journaliste financière) et Aleksandar (investisseur débutant).

Ce que nous avons livré

Entre janvier à mai 2026, nous avons intégralement construit, livré et déployé la plateforme. Elle couvre l'ensemble du pipeline, de la collecte brute jusqu'à l'interface utilisateur :

- **Pipeline ETL** multi-exchange (Binance, Kraken, Coinbase) avec bougies OHLCV, tickers, données CoinGecko et actualités RSS
- **API REST FastAPI** — 30+ endpoints couvrant toutes les fonctionnalités
- **Dashboard Streamlit** — 6 pages : chandelier, Market Overview, signaux BUY/SELL/HOLD, veille NLP, backtesting ML, paper trading
- **Moteur ML** — walk-forward backtesting, XGBoost/Random Forest/Régression Logistique, suivi MLflow (tracking distant DagsHub)
- **Paper trading temps réel** — WebSocket Binance, portefeuilles fictifs, P&L live
- **Infrastructure VPS** — Docker, Nginx, Ansible, Prometheus + Grafana 4 dashboards (stack versionnée dans `_v1/infra/`)
- **Déploiement cloud POC** — API sur Render (free tier), base PostgreSQL Supabase, frontend Streamlit Cloud (<https://nubxasqsgjdunghlifcfzn.streamlit.app/>), collecte quotidienne automatisée via GitHub Actions

Adéquation avec le sujet

L'intégralité du périmètre défini dans notre cadrage a été livrée. Les principaux ajustements techniques portent sur des choix délibérés : MongoDB remplacé par PostgreSQL (suffisant), BeautifulSoup par des flux RSS (plus fiable). L'apprentissage par renforcement figurait comme piste d'évolution optionnelle ; le paper trading constitue la première étape concrète vers cet objectif.

Points forts

La plateforme couvre une chaîne de valeur complète :

- collecte automatisée jusqu'aux signaux actionnables, tout est connecté en temps réel
- adoption dès le premier jour d'une CI/CD, des branches protégées et des tests automatiques — une rigueur d'ingénierie qui a évité la dette technique
- paper trading avec WebSocket Binance et infrastructure Ansible/Grafana (stack VPS)
- déploiement cloud POC opérationnel (Render + Supabase + Streamlit Cloud + DagsHub) avec collecte quotidienne automatisée et alertes email matin

Perspectives

Apprentissage par renforcement pour automatiser les décisions de trading, données on-chain, authentification JWT. La plateforme est accessible en POC via Render (API) et Streamlit Cloud (<https://nubxasqsgjdunghlifcfzn.streamlit.app/>) ; la prochaine étape est une mise en production complète sur VPS avec la stack Ansible/Nginx/Prometheus/Grafana déjà versionnée.

RAPPORT COMPLET

1. Contexte du projet

1.1 Sujet DataScientest

Le sujet **Crypto / Finance** nous demande de *"travailler en équipe pour faire ressortir de la valeur ajoutée, montrer les compétences pour définir un cahier des charges, une implémentation technique, et assurer la mise en production avec monitoring"*.

Le projet est structuré en trois grandes étapes pédagogiques :

1 — Cahier des charges 2 — Conception 3 — Mise en production

1.2 Objectif central

"Créer un bot de trading basé sur un modèle de Machine Learning qui investira sur les marchés crypto."

Nous avons interprété cet objectif de manière large : nous avons construit une **plateforme complète d'aide à la décision**, allant de la collecte de données jusqu'à la simulation de trading, avec toute la chaîne analytique et ML au milieu.

1.3 Planning initial

Date	Étape
03/09/2025	Kick-off
Septembre 2025	Discovery et préparation projet
Octobre 2025	Cahier des charges
Novembre 2025	MVP

Décembre 2025	Schéma BDD et pipeline
Février 2026	Intégration des algorithmes ML/DL
Avril 2026	Création de l'API
Mai 2026	Déploiement automatisé
Juin 2026	Rapport et Soutenance

2. Phase Discovery

2.1 Analyse du secteur

Notre phase de discovery nous a conduit à cartographier le secteur des crypto-monnaies selon plusieurs dimensions.

Le marché en chiffres :

Exchange	Volume journalier médian	Cryptos listées
Binance	16,3 Mds \$	517
Bybit	4,2 Mds \$	733
Crypto.com	3,6 Mds \$	419
OKX	2,6 Mds \$	343
Coinbase	2,5 Mds \$	298

Binance représente à lui seul plus de 40 % des volumes mondiaux sur les échanges centralisés. Nous avons retenu Binance, Kraken et Coinbase comme sources primaires — les trois échanges disposant à la fois d'une API ccxt robuste et d'une liquidité suffisante pour des données OHLCV fiables.

Popularité des actifs : Bitcoin est identifié par 90 % des répondants, Ethereum par 50 %, ce qui justifie notre focus sur les paires BTC/USDT et ETH/USDT comme paires de référence.

2.2 Personas utilisateurs

Nous avons identifié trois profils cibles à partir de notre travail de discovery :

Noah, 32 ans — Trader indépendant

Utilise des perpétuels sur DEX (Hyperliquid). Suit le marché quotidiennement, cherche une vue unifiée graphiques/actualités/signaux avec transparence sur les critères d'alerte. Son parcours idéal : signal → plan de trade → lien DEX. Il a besoin que la plateforme lui apprenne *pourquoi* un signal a été déclenché.

Sarah, 30 ans — Journaliste financière

Double formation généraliste et économie-finance, sans compétence technique. Cherche une veille fiable, filtrage du bruit, détection des fake news, alertes réglementaires, résumés automatiques. L'outil doit lui permettre de produire des articles précis et de développer rapidement une expertise.

Aleksandar — Investisseur débutant

Veut apprendre à investir en crypto sans risque réel, suivre un portefeuille fictif et comprendre les mécanismes des marchés. La pédagogie et la simplicité priment.

2.3 Benchmark concurrentiel

Nous avons identifié et analysé une dizaine de solutions existantes :

Solution	Positionnement	Fonctionnalités clés	API
Altfin	Trading analytics	Signaux techniques, screener	Oui
CryptoQuant	Analytics on-chain	Métriques on-chain, flux exchange	Oui
3commas.io	Trading bots	Paper trading, bots automatisés	Oui
Santiment	Data & sentiment	Sentiment de marché, données sociales	Oui
Glassnode	Analytics on-chain	Wallets, transactions, DeFi	Oui
Tickeron	Trading généraliste	Signaux, paper trading, crypto + finance tradi	Oui
Altrady	Trading multibourses	Data viz, paper trading, multi-exchange	Oui
Quantify Crypto	Data viz	Heatmaps, analytics	Oui
Dune Analytics	Data on-chain	SQL-like sur données blockchain	Oui

Enseignements du benchmark : Les solutions existantes sont soit coûteuses pour un usage individuel, souvent très spécialisées (uniquement on-chain, uniquement signaux techniques, ou uniquement trading automatisé). Aucune ne combine dans une même plateforme open-source la collecte ETL, l'analyse NLP de news, les signaux techniques, le backtesting ML et le paper trading.

2.4 Cadre réglementaire identifié

Notre veille réglementaire a identifié les textes et organismes clés encadrant notre activité :

- **MiCA** (Markets in Crypto-Assets) : règlement européen entré en application en 2024, encadrant les émetteurs de crypto-actifs et les prestataires de services
- **AMF** (Autorité des Marchés Financiers) : régulateur français, compétent sur les PSAN (Prestataires de Services sur Actifs Numériques)
- **ESMA** (European Securities and Markets Authority) : coordination réglementaire européenne
- **SEC** (Securities and Exchange Commission) : autorité américaine, influence les cours mondiaux par ses décisions (ETF, enforcement)
- **MiFID 2** : directive européenne sur les marchés financiers, applicable à certains instruments crypto
- **RGPD** : applicable à notre gestion des données utilisateurs (emails d'abonnés, adresses IP)

2.5 KPIs sectoriels identifiés

KPI	Définition
Market Capitalization	Prix courant × quantité en circulation
Total Value Locked	Valeur totale détenue sur un réseau DeFi — indicateur de vitalité
Volume journalier	Volume de transactions sur 24h — indicateur de liquidité
Fear & Greed Index	Indice de sentiment de marché (0-100)
RSI, MACD, BB ...	Indicateurs techniques classiques

Sharpe ratio	Rendement ajusté du risque (évaluation stratégies)
--------------	--

3. Équipe et organisation

3.1 Répartition des rôles

Notre cadrage estimait **70 jours de travail total** (40 DE + 30 DS) pour un budget théorique de **32 000 €** à 400 €/j TJM, auxquels s'ajoutent 4 000 € de coûts annexes (cloud, APIs, maintenance).

3.2 Méthode de travail

Nous avons adopté un workflow Git structuré dès le premier jour :

- **Branches protégées** : `main` (production) et `dev` (intégration)
- **Feature branches** : une branche par fonctionnalité, mergée via Pull Request
- **88+ Pull Requests** ouvertes et mergées sur la durée du projet
- **CI/CD** via GitHub Actions : tests automatiques sur chaque PR avant merge
- **Documentation** : README, docs/ maintenus en parallèle du code

4. Roadmap réalisée

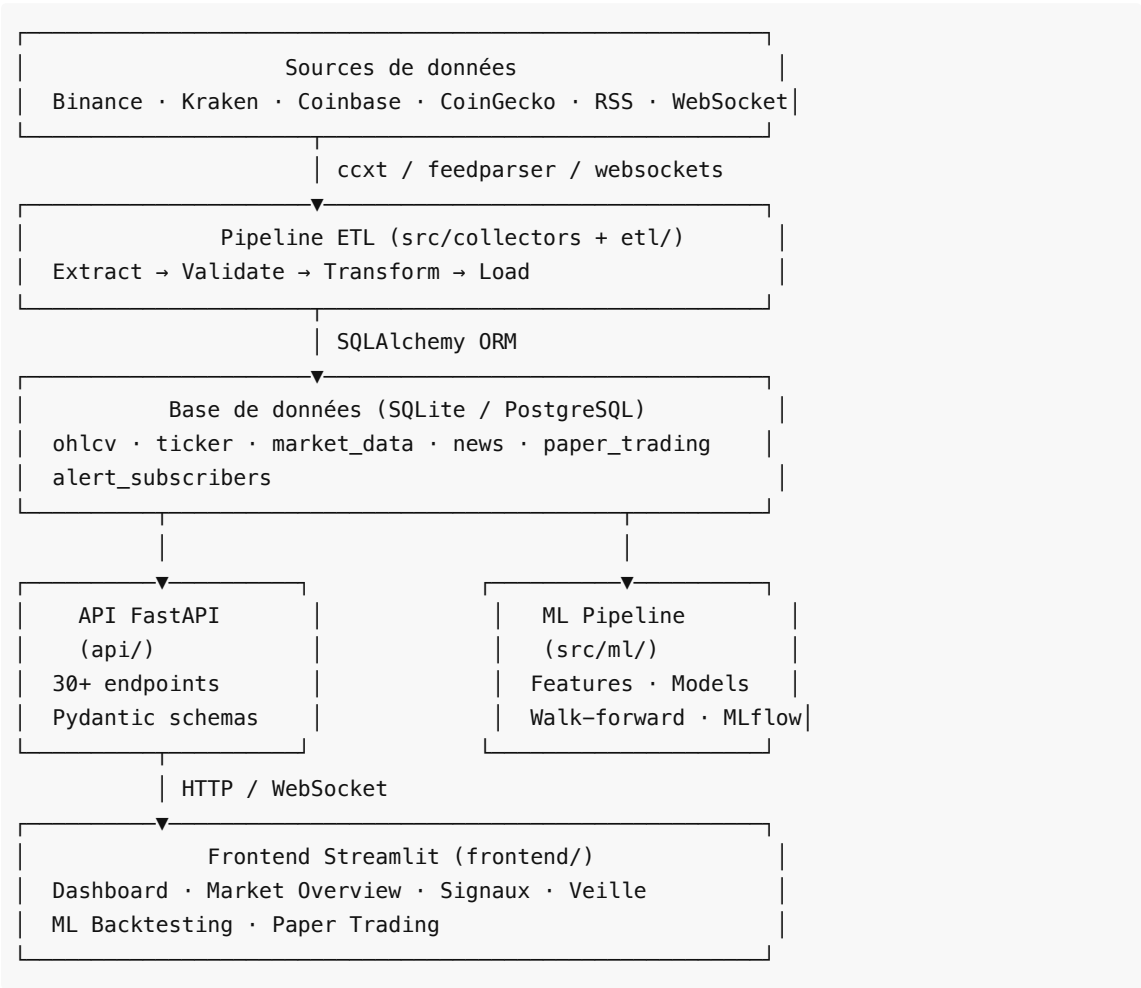
Voici notre réalisation effective :

Sprint	Thème prévu	Statut	Réalisation effective
1	DevOps & CI/CD	✓	GitHub Actions, branches protégées, Docker (livré dès jan 2026)
2	Collecte APIs (CoinGecko, news...)	✓	ccxt multi-exchange + CoinGecko + RSS feedparser
3	Structuration BDD	✓	SQLAlchemy + SQLite/PostgreSQL, 8 domaines de tables
4	ETL & qualité données	✓	Pipeline Extract→Validate→Transform→Load, DataValidator
5	Analytics & KPIs	✓	Indicateurs techniques (RSI, MACD, BB, SMA), Fear & Greed, signaux
6	ML supervisé	✓	XGBoost, RF, LR, walk-forward backtesting, MLflow — RL hors scope (piste d'évolution)
7	Dashboard v1	✓	6 pages Streamlit avec Plotly
8	Tests & CI/CD finalisation	✓	pytest, couverture src + api
9	Déploiement & monitoring	✓	Ansible, Nginx, Prometheus, Grafana
10	Alertes	✓	Alertes email livrées (démarrage collecte, fin collecte, erreurs critiques, abonnement)

11	Déploiement cloud gratuit	✓	Render (API), Supabase (PostgreSQL), Streamlit Cloud (frontend), DagsHub (MLflow), GitHub Actions collecte
----	---------------------------	---	--

5. Architecture technique

5.1 Vue d'ensemble



5.2 Stack technique : prévue vs retenue

Couche	Stack envisagée (cadrage)	Stack retenue	Justification
Collecte	requests, BeautifulSoup, python-binance	ccxt, feedparser	ccxt : multi-exchange unifié ; RSS : plus fiable que le scraping HTML
Sentiment/NLP	scikit-learn TF-IDF	scikit-learn TF-IDF + vaderSentiment	Conforme + ajout VADER pour scoring continu

Stockage	PostgreSQL + MongoDB	SQLAlchemy + SQLite/PostgreSQL + Supabase	MongoDB non nécessaire ; Supabase pour la prod cloud
API	FastAPI + Pydantic	FastAPI + Pydantic v2 + uvicorn	Conforme
Frontend	Streamlit ou Shiny	Streamlit + Plotly	Streamlit retenu
ML	scikit-learn, TensorFlow/PyTorch	scikit-learn + XGBoost	TF/PyTorch non justifiés sur notre volume de données
RL	Monte Carlo, SARSA, Q-learning	Hors scope	Piste d'évolution optionnelle ; paper trading = première étape
Experiment tracking	MLflow	MLflow + DagsHub	DagsHub pour le tracking distant en production
Infra VPS	Docker, GitHub Actions	Docker + GitHub Actions + Ansible + Nginx + Prometheus + Grafana	Stack versionnée dans _v1/infra/ — Prometheus/Grafana VPS uniquement
Infra cloud (POC)	—	Render + Supabase + Streamlit Cloud + DagsHub + GitHub Actions	Déploiement gratuit opérationnel ; Kraken par défaut (Binance bloqué côté US/GitHub)
Dev	—	Dev Container (.devcontainer)	Environnement de développement reproductible

5.3 APIs retenues

Parmi les APIs étudiées en phase de discovery, voici celles effectivement intégrées :

API	Usage	Gratuit
Binance (via ccxt)	OHLCV, ticker, WebSocket miniTicker	Oui
Kraken (via ccxt)	OHLCV	Oui
Coinbase (via ccxt)	OHLCV	Oui
CoinGecko	Fear & Greed, market cap, top movers	Oui (limité)
RSS multi-sources	Actualités crypto (feedparser)	Oui

Les APIs PhoenixNews, Cryptorank et CoinGecko Premium (identifiées comme pertinentes en discovery) n'ont pas été intégrées car payantes. L'axe de diversification des sources news reste un axe d'évolution.

5.4 Base de données

Notre schéma comporte **8 domaines de tables** :

Domaine	Tables	Description
OHLCV	ohlcv	Bougies multi-exchange, multi-timeframe
Ticker	ticker_snapshots	Snapshots périodiques prix/volume
Market Data	global_market_snapshot, top_crypto_snapshot, crypto_detail_snapshot	Données CoinGecko
News & NLP	news_articles	Articles enrichis (VADER, TF-IDF, entités, topics)
Alerting	alert_subscribers	Emails abonnés
Paper Trading	paper_portfolios, paper_trades	Portefeuilles et ordres fictifs

Le dual-support SQLite (dev) / PostgreSQL (prod) est géré via `CRYPTO_BOT_DB_URL` dans le Makefile.

6. Fonctionnalités réalisées

6.1 Pipeline de collecte de données

Architecture ETL modulaire (`src/collectors/` , `src/etl/`) :

- `Extractor` : appel API via ccxt avec gestion des rate limits
- `Transformer` : normalisation des timestamps (UTC), déduplication, validation des types, calcul d'indicateurs de base
- `Loader` : insertion incrémentale (upsert par clé composite exchange/symbol/timeframe/timestamp)
- `DataValidator` : contrôles de cohérence OHLCV (high $\geq$ low, volumes positifs, timestamps valides)

Sources et modes de collecte :

Source	Données	Mode
Binance, Kraken, Coinbase	OHLCV	Incrémental, planifié 09h00, historique 1000 bougies
Binance WebSocket	Prix live	Continu (daemon thread)
CoinGecko	Fear & Greed, market cap global, top movers	À la demande via API
RSS (multi-sources)	Actualités crypto	Une passe ou boucle 60 min

6.2 API REST

Notre API FastAPI expose **30+ endpoints** en 7 routeurs :

Routeur	Endpoints principaux
ohlcv	Bougies, liste des symboles disponibles
market	Fear & Greed, market cap global, top movers, historique
signals	Score composite RSI/MACD/BB/SMA par symbole
news	Articles filtrables (source, sentiment, topic, symbole)
ml	Backtest walk-forward, features ML
alerts	Subscribe/unsubscribe email, liste abonnés
paper_trading	Portefeuilles, ordres BUY/SELL, P&L, prix live

6.3 Frontend Streamlit — 6 pages

Page	Persona principal	Contenu
Dashboard	Noah	Chandelier Plotly + SMA/EMA/BB superposables
Market Overview	Sarah, Aleksandar	Fear & Greed, market cap, top movers, corrélations
Signaux	Noah	Score BUY/SELL/HOLD par actif (RSI/MACD/BB/SMA)
Veille	Sarah	News RSS enrichies NLP, filtres multi-critères, abonnement alertes
ML & Backtesting	Noah, DS	Walk-forward, Sharpe, PnL, drawdown, vs buy-and-hold
Paper Trading	Aleksandar, Noah	Portefeuilles fictifs, ordres live WebSocket, courbe capital

6.4 Machine Learning et Backtesting

Feature engineering : le `FeatureBuilder` génère automatiquement à partir des bougies OHLCV : SMA 7/14/21/50, EMA 7/14/21, RSI 14, MACD signal/histogramme, Bollinger Bands, volume relatif, log-returns sur 1/3/5 jours, label J+1.

Walk-forward backtesting : méthode rigoureuse anti-data-leakage avec purge et embargo entre fenêtres d'entraînement (défaut 60 jours) et de test (défaut 15 jours). Chaque fold est évalué indépendamment.

Modèles évalués :

Modèle	Type	Rôle
Dummy Classifier	Baseline probabiliste	Référence plancher
Régression Logistique	Baseline linéaire	Référence interprétable
Random Forest	Ensemble bagging	Robustesse

XGBoost	Gradient boosting	Modèle principal
----------------	-------------------	------------------

Métriques : Sharpe ratio annualisé, win rate, PnL total (log-returns), max drawdown, accuracy directionnelle, profit factor, comparaison vs buy-and-hold.

Chaque backtest est automatiquement loggé dans MLflow avec paramètres, métriques et artifacts.

6.5 NLP & Text Mining

Analyse	Méthode	Exemple
Sentiment	VADER (compound -1 → +1)	score: 0.72 → positif
Mots-clés	TF-IDF unigrammes + bigrammes	["etf approved", "sec", "rally"]
Entités	Regex + dictionnaire	{"crypto_symbols": ["BTC", "ETH"], "exchanges": ["binance"]}
Topics	Classification par mots-clés	["regulation", "adoption"]

8 topics : regulation , hack_security , adoption , defi , nft , macro , price_action , general .

6.6 Système d'alertes email

Notifications SMTP (Gmail) pour les événements clés :

- Démarrage et fin de collecte (avec résumé ETL + état de la base)
- Confirmation d'inscription (avec les 5 dernières actualités)
- Confirmation de désabonnement
- Alerte erreur critique pipeline

6.7 Paper Trading

Composants :

- PaperTrader : moteur métier (portefeuilles, P&L, gestion positions)
- LivePriceCache : singleton thread-safe alimenté par WebSocket Binance
- Rafraîchissement automatique toutes les 5s (streamlit-autorefresh)

7. Infrastructure et déploiement

7.1 Environnement local

```
make setup      # Installation multi-OS (setup.sh : macOS / Debian / RedHat / Arch / Alpine)
make run        # API + Streamlit (SQLite)
make run-all    # API + MLflow + Streamlit
make docker     # Stack complète Docker
```

7.2 Docker Compose

Stack Docker : API (FastAPI 8000), Frontend (Streamlit 8501), MLflow (5001 avec `--allowed-hosts "*"`).

7.3 Infrastructure production

Nous avons conçu et versionné une infrastructure de déploiement production complète (`_v1/infra/`) :

- **Ansible** : provision VPS vierge (packages, swap 2G, UFW, Fail2Ban), déploiement via rsync, sauvegardes quotidiennes, SSL Let's Encrypt
- **Nginx** : reverse proxy avec rate limiting (30 req/s API), support WebSocket Streamlit, headers de sécurité
- **Prometheus + Grafana** : 4 dashboards de monitoring — API overview, business metrics, PostgreSQL, ressources système — scraping toutes les 15s (stack VPS uniquement, non activée en cloud)
- **Services VPS** : TimescaleDB, MinIO (artifacts MLflow), API, Frontend, ETL worker, ML worker, Prometheus, Grafana

7.4 Déploiement cloud POC

En parallèle de la stack VPS, la plateforme est accessible via un déploiement cloud gratuit :

- **Render** (free tier) — API FastAPI, déployée en continu via `render.yaml`
- **Supabase** — PostgreSQL managé en production
- **Streamlit Cloud** — frontend accessible à `https://nubxasqsgjdunghlifcfzn.streamlit.app/`
- **DagsHub** — MLflow tracking distant
- **GitHub Actions** — collecte quotidienne automatisée (Kraken par défaut, Binance bloqué sur serveurs US) avec alertes email de démarrage et de fin chaque matin

Note : Prometheus et Grafana ne sont pas utilisés dans ce déploiement cloud free tier — ils font partie de la stack VPS versionnée dans `_v1/infra/` .

7.5 CI/CD

GitHub Actions déclenche les tests sur chaque Pull Request. Nous avons protégé les branches `main` et `dev` contre les pushes directs dès février 2026.

8. Qualité et tests

Notre suite de tests couvre **20 fichiers**, **~150 classes de test** et **près de 450 fonctions**, organisés par domaine fonctionnel. Chaque Pull Request déclenche automatiquement cette suite en CI via GitHub Actions.

```
make test-cov # pytest --cov=src --cov=api --cov-report=term-missing
```

8.1 ETL — pipeline de données

Fichier	Ce qui est testé
<code>test_etl_extractor.py</code>	Initialisation de l' <code>OHLCEExtractor</code> , extraction unitaire et multi-exchange, gestion des erreurs ccxt
<code>test_etl_transformer.py</code>	Normalisation timestamps, déduplication, validation des types, traitement batch

test_etl_loader.py	Upsert en base, insertions batch, opérations sur tables, gestion des conflits de clé
test_etl_pipeline.py	Pipeline complet Extract→Transform→Load, PipelineResult, opérations batch, résumé d'exécution
test_data_validator.py	Validation structure DataFrame OHLCV, colonnes prix (high ≥ low, close entre high/low), volumes positifs, cohérence prix, métadonnées

8.2 Collecteurs

Fichier	Ce qui est testé
test_ohlcw_collector.py	Initialisation, validation des paramètres, collecte complète avec stockage
test_fear_greed_collector.py	Parsing de la réponse CoinGecko, fetch HTTP, context manager
test_news_collector.py	Analyse de sentiment VADER, extraction de mots-clés TF-IDF, parsing d'entrées RSS, collecte et stockage d'articles, sources par défaut
test_ticker_service.py	Cache ticker thread-safe (TickerCache), collecteur WebSocket (TickerCollector), interactions base de données

8.3 Machine Learning

Fichier	Ce qui est testé
test_feature_builder.py	Validation entrées, sorties attendues, features de retour (log-returns 1/3/5j), volatilité, indicateurs techniques (RSI, MACD, BB, SMA, EMA), structure des bougies, volume relatif, features temporelles
test_dataset_builder.py	Calcul de la direction (label J+1), calcul des retours, découpage temporel (TimeSeriesSplit), split train/test sans fuite temporelle
test_baseline.py	Initialisation des modèles baseline, fit, predict, predict_proba, cross-validation, feature importances, DummyClassifier
test_backtester.py	Calcul du Sharpe ratio, profit factor, max drawdown, initialisation, validation des paramètres, walk-forward complet, métriques agrégées, comparaison vs baseline
test_evaluator.py	Évaluation des folds, folds avec retours financiers, comparaison multi-modèles, statistiques financières (PnL, win rate)

8.4 API REST

test_api.py — tests des endpoints FastAPI via `httpx.AsyncClient` sur base SQLite en mémoire :

Classe	Endpoints couverts
TestHealth	GET /health
TestOHLCV	GET /ohlcw/ — filtres exchange, symbol, timeframe, pagination

TestOHLCVSymbols	GET /ohlcv/symbols
TestOHLCVLatest	GET /ohlcv/latest
TestMarketTop	GET /market/top
TestMarketGlobal	GET /market/global
TestMarketTicker	GET /market/ticker
TestSignals	GET /signals/ — vérification scores et direction BUY/SELL/HOLD

8.5 Paper trading

`test_paper_trading.py` — tests du moteur `PaperTrader` et des endpoints dédiés :

- Récupération du dernier prix via `LivePriceCache`
- Création de portefeuille, ouverture et fermeture de position
- Calcul du P&L et résumé du portefeuille
- Endpoints REST : liste des portefeuilles, passage d'ordre, liste des ordres, cas limites (solde insuffisant, symbole inconnu)

8.6 Frontend

Fichier	Ce qui est testé
<code>test_frontend_api_client.py</code>	Client HTTP Streamlit (<code>APIClient</code>), <code>fetch_ohlcv</code> , <code>fetch_signals</code> , <code>fetch_market_global</code> , <code>fetch_market_top</code> , <code>fetch_symbols</code> — avec mock des appels réseau
<code>test_frontend_components.py</code>	Calcul des croisements MACD, rendu du chandelier Plotly, <code>safe_float</code> , labels RSI/BB/MACD
<code>test_frontend_utils.py</code>	Extraction des timeframes et symboles disponibles, formatage des timestamps

8.7 Configuration et fixtures

`conftest.py` fournit une fixture de session `setup_test_config` qui initialise une configuration de test minimale (SQLite en mémoire, paires BTC/USDT-ETH/USDT, scheduler désactivé) sans dépendances externes, ce qui permet d'exécuter l'ensemble de la suite sans fichier `.env` ni connexion réseau.

9. Adéquation avec le sujet DataScientest

9.1 Livrables produits

Livrable attendu	Statut	Forme
Guide d'entretien / Discovery	✓	Glossaire secteur, benchmark, personas (XLSX + cadrage)
Experience Map	✓	3 personas détaillés (Noah, Sarah, Aleksandar)
Analyse SWOT	✓	SWOT sectoriel crypto

Veille technologique	✓	Benchmark 10 concurrents, APIs étudiées, réglementation
Schéma BDD	✓	docs/database_schema.md
Pipeline ETL	✓	src/etl/ — Extract → Validate → Transform → Load
MVP fonctionnel	✓	6 pages Streamlit + API REST + ML
Roadmap technique	✓	10 sprints documentés
Tests	✓	pytest, CI GitHub Actions
Déploiement & monitoring	✓	Ansible, Docker, Nginx, Prometheus, Grafana (VPS) + Render/Streamlit Cloud (cloud POC)
Rapport final	✓	Ce document

9.2 Fonctionnalités du sujet

Feature cadrage	Statut	Notes
Veille actu & marché	✓ Livré	RSS + VADER + NLP, au-delà du scope
Suivi des cours temps réel	✓ Livré	WebSocket Binance, non prévu initialement
Dashboard	✓ Livré	Chandelier interactif + indicateurs
Analytics	✓ Livré	Market Overview (Fear & Greed, corrélations...)
ML supervisé	✓ Livré	Random Forest, XGBoost, walk-forward
ML non supervisé	⚠ Partiel	Pas de clustering explicite en production (hors scope initial)
ML par renforcement	— Hors scope	Piste d'évolution optionnelle ; paper trading = première étape concrète
Paper trading	✓ Livré et élargi	WebSocket live, métriques financières complètes
Monitoring infra	✓ Livré	Prometheus + Grafana (stack VPS — non activé en cloud free tier)
PostgreSQL	✓ Livré	+ migration SQLite→PostgreSQL
MongoDB	✗ Abandonné	PostgreSQL suffisant
TensorFlow/PyTorch	✗ Non utilisés	scikit-learn + XGBoost suffisants
MLflow	✓ Livré	Conforme
Docker	✓ Livré	API + Frontend + MLflow
GitHub Actions CI/CD	✓ Livré	Dès le départ

10. Analyse rétrospective

10.1 Points forts

Complétude de la chaîne de valeur : nous livrons une plateforme end-to-end fonctionnelle, de la collecte brute jusqu'au trading simulé. Aucun maillon n'est laissé en suspens.

Rigueur dès le départ : CI/CD, branches protégées, tests unitaires et architecture modulaire ont été mis en place dès le premier commit. Cette rigueur a évité la dette technique.

Au-delà du scope du cadrage : WebSocket Binance pour les prix temps réel, paper trading complet avec P&L live, infrastructure Ansible/Prometheus/Grafana (VPS), `setup.sh` multi-OS, Dev Container, déploiement cloud POC (Render + Supabase + Streamlit Cloud) et catalogue UML V2 (22 diagrammes) dépassent les ambitions initiales.

~88 Pull Requests en 7 mois : collaboration structurée avec répartition claire des responsabilités.

10.2 Difficultés rencontrées

MLflow en Docker : le middleware de sécurité de MLflow 2.x rejette par défaut les requêtes ne venant pas de `localhost`. L'ajout de `--allowed-hosts "*"` a résolu le problème.

Installation multi-OS (`setup.sh`) : la plateforme devait fonctionner sur les machines des deux membres de l'équipe (macOS et windows), les environnements CI Linux et le VPS Debian de production. Un simple `pip install -r requirements.txt` ne suffisait pas : `psycpg2` requiert `libpq-dev` sur Debian, `postgresql-devel` sur RedHat, ou `libpq` via Homebrew sur macOS ; plusieurs packages ML nécessitent en outre des headers de compilation (`python3-dev` , `build-essential`). Nous avons écrit `setup.sh` , un script bash qui détecte l'OS via `uname -s` puis affine la distribution Linux via les marqueurs `/etc/debian_version` , `/etc/redhat-release` , `/etc/arch-release` et `/etc/alpine-release` , avant d'appeler le bon gestionnaire de paquets (`apt` , `yum` , `pacman` , `apk` , `brew`). Le script vérifie également la version Python (3.10+ requis), crée et active le venv (sauf si conda est déjà actif), copie `.env.example` en `.env` et initialise les dossiers nécessaires. Windows est explicitement détecté et orienté vers WSL2 ou `make docker` . Ce script a uniformisé l'onboarding et supprimé les divergences d'environnement entre les membres de l'équipe et le CI.

Compatibilité Linux : `make run` présentait un bug sur Linux (absence de timeout sur la boucle d'attente de l'API, erreurs uvicorn masquées). Résolu par un compteur de timeout et `--log-level info` .

Cache Streamlit : `@st.cache_resource` a causé une `AttributeError` lors des mises à jour de code sans redémarrage complet — comportement inhérent à Streamlit.

Volume de données pour le ML : le walk-forward nécessite ~260 bougies minimum. Le rate limiting de CoinGecko et des échanges a rendu la collecte initiale laborieuse.

10.3 Axes d'amélioration

Apprentissage par renforcement : le paper trading peut servir d'environnement de simulation pour un agent RL (Monte Carlo, Q-learning comme prévu dans la roadmap).

Authentification : la plateforme ne gère pas encore les utilisateurs.

Sources news premium : PhoenixNews (~1500 sources temps réel) et Cryptorank ont été identifiés en discovery mais non intégrés (coût).

Mise en production complète : l'API est déployée sur Render avec Supabase ; ce déploiement cloud gratuit constitue une vitrine fonctionnelle.

Frontend cloud : le déploiement Streamlit sur Render est en cours de finalisation (configuration en place dans `render.yaml`).

Annexes

A. Chronologie des commits

Période	Activité principale
Jan 2026	Fondations ETL, SQLite, tests, CI/CD
Fév 2026	Market data CoinGecko, tables supplémentaires
Mar 2026	ML feature engineering, modèles baseline
Avr 2026	Sprint : API, Frontend, MLflow, Backtesting
Mai 2026	Docker, NLP, signaux, XGBoost, alerting
Mai 2026	Paper trading, PostgreSQL, WebSocket
Mai 2026	Fixes, setup, docs, infra prod
Mai–Jun 2026	Déploiement cloud Render/Supabase/DagsHub, Dev Container, catalogue UML V2 (22 diagrammes), Kraken par défaut, collecte quotidienne GitHub Actions
Total	~100+ commits, 88 PRs

B. Livrables

Livable	Localisation
Code source	Marivel75/_Crypto_Bot (GitHub)
API documentée (local)	<code>http://localhost:8000/docs</code>
API (cloud)	Render — voir <code>render.yaml</code>
Front (local)	<code>http://localhost:8501</code>
Documentation technique	<code>docs/</code>
Schéma BDD	<code>docs/database_schema.md</code>
Diagrammes UML V2 (22)	<code>docs/diagrams/</code>
Infrastructure prod	<code>_v1/infra/</code>
Script d'installation	<code>setup.sh</code>
Dev Container	<code>.devcontainer/devcontainer.json</code>
Tests	<code>tests/</code>

Cadrage	Crypto_bot_cadrage_V2.pdf
Ressources discovery	ressources_projet/
Rapport final	ressources_projet/rapport_final.md

C. Dépendances principales

```

ccxt>=4.0.0           # Multi-exchange crypto
FastAPI>=0.104.0       # API REST
SQLAlchemy>=2.0.0      # ORM
streamlit>=1.40.0      # Frontend
plotly>=5.18.0         # Visualisation
scikit-learn>=1.3.0    # ML
xgboost>=2.0.0         # Gradient boosting
mlflow>=2.10.0         # Experiment tracking
vaderSentiment>=3.3.2  # Analyse de sentiment
feedparser>=6.0.0      # Collecte RSS
websockets>=11.0       # WebSocket Binance
psycopg2-binary>=2.9.9 # PostgreSQL

```